

Objective:

Implement fuzzy inference systems using pure Python (without external libraries) to solve real-life decision-making problems. The systems use fuzzy logic concepts such as membership functions, fuzzy rules, fuzzification, inference, and centroid defuzzification.

Problem 1: Restaurant Tipping System

Problem Description: This fuzzy system determines the final tip percentage in a restaurant based on the quality of service and food. Since tipping depends on human judgment rather than exact values, fuzzy logic is suitable for modeling this uncertainty.

Membership Functions:

- Triangular for simple linear transitions
- Trapezoidal for stable middle ranges
- Gaussian for smooth and natural behavior

Code: Membership Functions

```
def triangular(x, a, b, c):
    if x <= a or x >= c:
        return 0.0
    elif a < x < b:
        return (x - a) / (b - a)
    elif b < x < c:
        return (c - x) / (c - b)
    else:
        return 1.0

def trapezoidal(x, a, b, c, d):
    if x <= a or x >= d:
        return 0.0
    elif b <= x <= c:
        return 1.0
    elif a < x < b:
        return (x - a) / (b - a)
    else:
        return (d - x) / (d - c)

def gaussian(x, mean, sigma):
    return math.exp[-0.5 * ((x - mean) / sigma) ** 2]
```

Fuzzification

Crisp inputs are converted into fuzzy values using membership functions.

Code: Fuzzification

```
def service_membership(service):  
    poor = triangular(service, 0, 0, 5)  
    good = trapezoidal(service, 3, 5, 7, 9)  
    excellent = gaussian(service, 9, 1)  
    return poor, good, excellent  
  
def food_membership(food):  
    bad = triangular(food, 0, 0, 5)  
    delicious = gaussian(food, 8, 1.5)  
    return bad, delicious
```

Fuzzy Rules

Six fuzzy rules are defined to cover all service–food combinations.

- IF service is poor AND food is bad → tip is cheap
- IF service is poor AND food is delicious → tip is cheap
- IF service is good AND food is bad → tip is average
- IF service is good AND food is delicious → tip is average
- IF service is excellent AND food is bad → tip is generous
- IF service is excellent AND food is delicious → tip is generous

Code: Inference

```
def inference(service, food):
    s_poor, s_good, s_excellent = service_membership(service)
    f_bad, f_delicious = food_membership(food)
    cheap = max(
        min(s_poor, f_bad),
        min(s_poor, f_delicious)
    )
    average = max(
        min(s_good, f_bad),
        min(s_good, f_delicious)
    )
    generous = max(
        min(s_excellent, f_bad),
        min(s_excellent, f_delicious)
    )
    return cheap, average, generous
```

Defuzzification

Centroid defuzzification converts fuzzy output into a crisp tip percentage.

Code: Defuzzification

```
def defuzzify(cheap, average, generous):
    num = 0.0
    den = 0.0
    for tip in range(5, 26):
        c = min(cheap, triangular(tip, 5, 10, 15))
        a = min(average, trapezoidal(tip, 10, 13, 17, 20))
        g = min(generous, gaussian(tip, 22, 2))

        agg = max(c, a, g)
        num += tip * agg
        den += agg
    if den != 0:
        return num / den
    else:
        return 0
```

Testing

```
if __name__ == "__main__":
    service = 3
    food = 6
    c, a, g = inference(service, food)
    tip = defuzzify(c, a, g)
    print("Final Tip Percentage:", round(tip, 2))
```

Output: Final Tip Percentage: 10.0

Colab Link: <https://colab.research.google.com/drive/1AqTxuj-l-nKil3ilWlclK1lR-a1aNjCt?usp=sharing>

Problem 2: Fuzzy Washing Machine Controller

Problem Description: This fuzzy system controls the wash time of a washing machine based on the dirtiness of clothes and the load size. Fuzzy logic allows flexible control instead of fixed wash durations.

Membership Functions

The same three membership functions are reused to model dirtiness, load size, and wash time.

```
def triangular(x, a, b, c):
    if x <= a or x >= c:
        return 0.0
    elif a < x < b:
        return (x - a) / (b - a)
    elif b < x < c:
        return (c - x) / (c - b)
    else:
        return 1.0

def trapezoidal(x, a, b, c, d):
    if x <= a or x >= d:
        return 0.0
    elif b <= x <= c:
        return 1.0
    elif a < x < b:
        return (x - a) / (b - a)
    else:
        return (d - x) / (d - c)

def gaussian(x, mean, sigma):
    return math.exp(-0.5 * ((x - mean) / sigma) ** 2)
```

Code: Fuzzification

```
def dirtiness_membership(d):
    low = triangular(d, 0, 0, 40)
    medium = trapezoidal(d, 30, 45, 55, 70)
    high = gaussian(d, 80, 10)
    return low, medium, high

def load_membership(l):
    small = triangular(l, 0, 0, 40)
    medium = trapezoidal(l, 30, 45, 55, 70)
    large = gaussian(l, 80, 10)
    return small, medium, large
```

Fuzzy Rules

Six rules determine wash time based on dirtiness and load size.

- IF dirtiness is low AND load is small → wash time is short
- IF dirtiness is low AND load is medium → wash time is short
- IF dirtiness is medium AND load is small → wash time is medium
- IF dirtiness is medium AND load is medium → wash time is medium
- IF dirtiness is high AND load is medium → wash time is long
- IF dirtiness is high AND load is large → wash time is long

```
def inference(dirt, load):
    d_low, d_med, d_high = dirtiness_membership(dirt)
    l_small, l_med, l_large = load_membership(load)

    short = max(
        min(d_low, l_small),
        min(d_low, l_med)
    )
    medium = max(
        min(d_med, l_small),
        min(d_med, l_med)
    )
    long = max(
        min(d_high, l_med),
        min(d_high, l_large)
    )
    return short, medium, long
```

Defuzzification: Centroid method computes final wash time.

Code: Defuzzification

```
def defuzzify(short, medium, long):
    num = 0.0
    den = 0.0

    for t in range(0, 101):
        s = min(short, triangular(t, 0, 20, 40))
        m = min(medium, trapezoidal(t, 30, 45, 55, 70))
        l = min(long, gaussian(t, 80, 10))

        agg = max(s, m, l)
        num += t * agg
        den += agg
    if den != 0:
        return num / den
    else:
        return 0
```

Testing (5 Cases)

```
if __name__ == "__main__":
    tests = [(20, 20), (40, 50), (80, 90), (60, 30), (30, 85)]
    for i, (d, l) in enumerate(tests, 1):
        s, m, lg = inference(d, l)
        wt = defuzzify(s, m, lg)
        print(f"Case {i}: Wash Time = {round(wt, 2)}")
```

Output:

Case 1: Wash Time = 20.0

Case 2: Wash Time = 50.02

Case 3: Wash Time = 79.38

Case 4: Wash Time = 50.0

Case 5: Wash Time = 64.24

Colab Link: <https://colab.research.google.com/drive/1AqTxuj-l-nKil3ilWlcIK1lR-a1aNjCt?usp=sharing>

Discussion and Conclusion:

Both fuzzy systems successfully demonstrate how uncertainty and human reasoning can be handled using fuzzy logic. The use of different membership functions improves flexibility and realism, while centroid defuzzification ensures smooth and meaningful outputs.